西北大学

第二章：Python 科学计算

谭振宇

西北大学城市与环境学院

2026 年 05 月 06 日

多维数组（NumPy）

二维表（Pandas）

Matplotlib 绘图库

SciPy 科学计算

Scikit-learn 机器学习

Scikit-image 图像处理

本章小结

为什么要学习 NumPy?

NumPy 是 Python 科学计算的基石，几乎所有科学计算库都构建在它之上。

核心价值

- 科学观测数据大多可抽象为**多维数组**
- 提供高效的向量化运算，远快于纯 Python
- Pandas、SciPy、Matplotlib 均构建于其上

安装方法

```
conda install -c conda-forge numpy
```

导入惯例：

```
import numpy as np
```

NumPy 的核心是多维数组 `ndarray`。整个 Python 科学计算体系都是建立在 `ndarray` 之上的。

ndarray 核心概念

ndarray 是 NumPy 中的多维数组对象，有三个重要属性：

ndim

维度数：数组的轴数（维数）。

shape

形状：一个元组，表示每个维度中的元素数目。

dtype

数据类型：数组元素的底层存储类型，如 int64、float32。

核心概念：维度的理解

把多维数组看成**一维数组的嵌套**：

- **第一个维度**：最外层的元素
- **最后一个维度**：最内层的元素

例如 shape=(2,3,4) 的数组：最外层有 2 块，每块有 3 行，每行有 4 个元素。

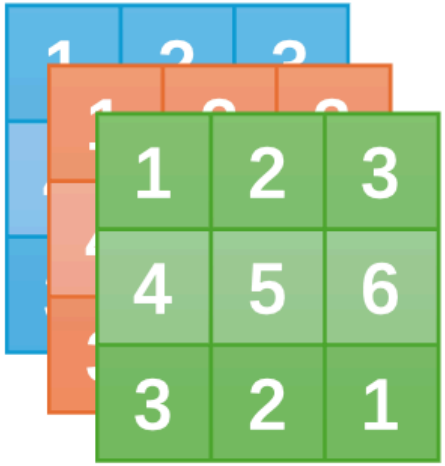
ndarray 核心概念

一维数组

```
ndim: 1
shape: (3,)
dtype: str
```

二维数组

```
ndim: 2
shape: (3,3)
dtype: float
```



三维数组

```
ndim: 3
shape: (3,3,3)
dtype: int
```

思考一下我们地理信息领域有哪些数据可以表示为多维数组?

数组维度理解

```
data = np.random.randint(10, size=(4, 3, 2))
```

数组的维度

```
[[[1 3]
  [7 8]
  [5 4]]

 [[5 1]
  [1 5]
  [1 0]]

 [[9 6]
  [7 7]
  [2 9]]

 [[1 6]
  [7 2]
  [2 8]]]
```

这是一个三维数组

第一个维度是最外层红色标注的4个元素

第二个维度是每个红色标注元素内部的3个绿色元素

第三个维度是每个绿色标注元素内部的2个蓝色元素

第一个维度对应最外层元素，最后个维度对应最里层元素

所以对于二维数组，第一维度是行，第二维度是列

维度理解对于掌握数组索引、切片、合并等操作至关重要

创建多维数组

西北大学

方法一：从列表创建

```python
import numpy as np
nums = np.array([[1, 2, 3], [4, 5, 6]])
print(nums.dtype)   # int64
print(nums.shape)   # (2, 3)
print(nums.ndim)    # 2
```

方法二：创建空数组并赋值

```python
nines = np.empty((3, 3), np.int64)
nines[:] = 9  # 所有元素赋值为 9
```

方法三：使用 range 生成并 reshape

```python
nums = np.reshape(range(16), (4, 4))
# 转置
nums = np.transpose(nums)    # 等价于 nums.T
# 再次 reshape 成三维
nums3d = np.reshape(nums, (4, 2, 2))
```

数组索引与切片

通过 start:stop:step 语法对每个维度进行切片：

```
nums = np.arange(12).reshape(4, 3)
# [[ 0  1  2]  [ 3  4  5]  [ 6  7  8]  [ 9 10 11]]

nums[0]         # 第一行: [0 1 2]
nums[:, 1]      # 第二列: [ 1  4  7 10]
nums[0, 1]      # 第一行第二列: 1
nums[-1, -2]    # 最后一行倒数第二列: 10
nums[[0, 2], :] # 取出第一和第三行
```

三维数组切片

```
nums = np.arange(18).reshape(3, 3, 2)
nums[1:, :, 0]    # 第一维第2个起，第三维取第1个
nums[0, :2, -1]   # 第一维第1个，第二维前2个，第三维最后1个
nums[:2, ...]     # 省略号代替后续所有维度
```

数组的基本操作

西北大学

算术运算（逐元素）

```
x + y    # 逐元素相加
x * y    # 逐元素相乘
x + 2    # 广播：每个元素加 2
x ** 2   # 逐元素平方
```

线性代数

```
x.dot(y)            # 向量内积
np.matmul(x, y)     # 矩阵乘法
np.linalg.det(x)    # 行列式
np.linalg.inv(x)    # 矩阵逆
x.T                 # 转置
```

元素筛选

```
np.where(nums > 5)        # 返回满足条件的索引
np.extract(nums > 5, nums) # 返回满足条件的元素
nums[nums > 5] = 5        # 条件赋值
```

排序

```
np.sort(nums, axis=0)    # 沿第一维排序
np.argsort(nums)         # 返回排序后的索引
nums.max(0)              # 第一维的最大值
np.argmax(nums, 0)       # 最大值的索引
```

数组的合并与分割

西北大学

vstack / hstack：直观拼接

```
np.vstack((x, y))  # 沿行方向（垂直）合并
np.hstack((x, y))  # 沿列方向（水平）合并
```

stack：新增维度合并

```
# 10 个 3×4 的数组，沿不同维度 stack
np.stack(data, axis=0).shape  # (10, 3, 4)
np.stack(data, axis=1).shape  # (3, 10, 4)
np.stack(data, axis=2).shape  # (3, 4, 10)
```

concatenate：在已有维度合并（维度不变）

```
np.concatenate(data, axis=0).shape  # (30, 4)
np.concatenate(data, axis=1).shape  # (3, 40)
# axis=0 等价于 vstack; axis=1 等价于 hstack
```

stack 合并后会**新增一个维度**；concatenate 合并后**维度数不变**。

增减维度：newaxis 与 squeeze

在处理遥感影像等数据时，经常需要动态增减数组维度：

```python
data = np.random.random((3, 4))  # shape: (3, 4)

# 新增一个维度（推荐这种写法，表意清晰）
data = data[np.newaxis, ...]
print(data.shape)   # (1, 3, 4)

# 再新增一个维度（等价写法）
data = data[None, ...]
print(data.shape)   # (1, 1, 3, 4)

# 去除大小为 1 的多余维度
data = np.squeeze(data)
print(data.shape)   # (3, 4)
```

向量化编程

关键原则：处理 NumPy 数组时，**尽量避免 for 循环**，使用向量化编程可大幅提升效率。

❌ 使用 for 循环（慢）

```python
# 10000×10000 数组，约需 1 分钟
for i in range(count):
    for j in range(count):
        v[i][j] = v[i][j]**2 + 3.14
```

✅ 向量化编程（快）

```python
# 同样的操作，约需 1.3 秒
vector = vector ** 2 + 3.14
```

沿指定维度进行向量化操作

```python
# 在第二维找最大值（apply_along_axis）
np.apply_along_axis(np.max, 1, data)
# 在第一维和第三维求和（apply_over_axes）
np.apply_over_axes(np.sum, data, (0, 2))
```

MaskedArray：处理无效数据

在遥感影像处理中，常用 MaskedArray 屏蔽云污染、无效像素等：

```python
from numpy import ma

data = np.random.randn(3, 4)

# 将小于 0 的元素掩膜掉（输出中显示为 --）
x = ma.MaskedArray(data, mask=(data < 0))

# 将大于 1 的元素掩膜掉
y = ma.MaskedArray(data, mask=(data > 1))

# 掩膜位置会在运算结果中自动传播
print(x + y)  # 任意一方被掩膜的位置，结果也被掩膜
```

若参与运算的任意数组中某位置的元素为**无效值（掩膜）**，则运算结果中该位置的元素也将自动变为**无效值**。

NumPy 核心知识小结

本节要点

- ndarray 的三个关键属性：ndim（维数）、shape（形状）、dtype（数据类型）
- **维度理解**：第一个维度为最外层，最后一个维度为最内层
- **索引和切片**：start:stop:step，多维用逗号分隔，省略号代替连续维度
- **合并**：concatenate（维度不变）vs stack（新增一维）
- **向量化编程**：避免循环，效率提升数十倍
- **MaskedArray**：处理无效数据，掩膜自动传播

多维数组（NumPy）

二维表（Pandas）

Matplotlib 绘图库

SciPy 科学计算

Scikit-learn 机器学习

Scikit-image 图像处理

本章小结

Pandas 简介

Pandas 是 Python 中最常用的数据处理与分析库，广泛应用于数据科学和机器学习领域。

核心数据结构

- `DataFrame`：二维表格，类似 Excel 或 SQL 表，每列可以是不同类型
- `Series`：一维数组，类似表格中的一列

核心功能

- 读写 CSV、Excel、SQL、JSON 等多种文件格式
- 数据过滤、分组、聚合、合并、连接
- 数据清洗：去重、填补缺失值

```
conda install -c conda-forge pandas
```

```
import pandas as pd
```

DataFrame 创建

西北大学

从列表（二维数组）创建

```python
data = [[1, 2, 3],
        [4, 5, 6],
        [7, 8, 9]]
df = pd.DataFrame(data,
    index=[1, 2, 3],
    columns=['a', 'b', 'c'])
```

从字典创建

```python
data = {
    'a': [4, 5, 6],
    'b': [7, 8, 9],
    'c': [10, 11, 12]
}
df = pd.DataFrame(data,
    index=[1, 2, 3])
```

从文件读取

```python
df = pd.read_csv('iris.csv')   # 读取 CSV
df = pd.read_excel('data.xlsx') # 读取 Excel
df.head(5)   # 查看前 5 行
df.tail(5)   # 查看后 5 行
df.shape     # 输出行数和列数
```

DataFrame 访问：列与行条件

Pandas 提供多种方式灵活访问数据：

```python
# 按列名选择单列（返回 Series）
data = df['petallength']
# df['petallength'] 与 df.petallength 等价

# 按列名选择多列（返回 DataFrame）
data = df[['petallength', 'petalwidth']]

# 按行条件筛选（萼片宽度 > 3.5 的所有行）
data = df[df.sepalwidth > 3.5]

# 多条件组合筛选（& 表示且，| 表示或）
data = df[(df.sepalwidth > 4) & (df.petalwidth < 0.5)]
```

选择单列时，返回类型为 Series；选择多列或按条件筛选时，返回类型为 DataFrame。

DataFrame 访问：iloc 与 loc

西北大学

iloc：基于整数位置（行列号）

```python
# 选择第 10 到 15 行
df.iloc[10:16]

# 选择第 2、3 列的所有行
df.iloc[:, [1, 2]]

# 访问具体某个元素（行号，列号）
df.iat[4, -1]
```

loc：基于标签（列名）

```python
# 按列名选择指定列
df.loc[:, ['sepalwidth', 'petalwidth']]

# 条件筛选行 + 指定列
df.loc[df.sepalwidth > 3.5,
       ['sepalwidth', 'class']]

# 访问具体某个元素（行号，列名）
df.at[4, 'class']
```

DataFrame 合并：concat

西北大学

按行拼接（axis=0，默认）

```python
# 两表结构相同，行合并（行数之和）
df = pd.concat([df1, df2])
```

按列拼接（axis=1）

```python
# 两表行数相同，列合并（列数之和）
df = pd.concat([df1, df2], axis=1)
```

增加单行或单列

```python
# 通过 loc 给 DataFrame 增加一行
df.loc[len(df.index)] = [43, 'M', 'Jay', 'NW']

# 直接通过 [] 给 DataFrame 添加新列
df['ID'] = df.Age * random.randint(0, 100)
```

DataFrame 合并：merge

merge() 实现类似 SQL 的表连接功能，通过 how 参数指定连接类型：

内连接（inner，默认）

保留**两表公共**元素，类似集合求**交集**。

外连接（outer）

保留**所有**元素，类似集合求**并集**，缺失值填 NaN。

左连接（left）

完全保留**左表**的所有行，右表无匹配则填 NaN。

右连接（right）

完全保留**右表**的所有行，左表无匹配则填 NaN。

```python
# on 参数指定两表中共有的基准列
pd.merge(df1, df2, how='inner', on='x1')  # 内连接
pd.merge(df1, df2, how='left',  on='x1')  # 左连接
pd.merge(df1, df2, how='outer', on='x1')  # 外连接
```

Pandas 常用工具函数

西北大学

数据清洗

```
df.drop_duplicates()  # 去重（按行）
df.dropna()           # 去除含空值的行
df.fillna(0)          # 将空值填充为 0
```

宽表 ↔ 长表转换

```
# 宽表 → 长表（列名变为行数据）
pd.melt(df, id_vars=['ID'],
        value_vars=['Name', 'Role'])

# 长表 → 宽表（行数据变为列名）
df.pivot(index='ID',
         columns='variable',
         values='value')
```

在使用基于 **Grammar of Graphics** 语法的绘图库（如 `plotnine`）时，通常需要将数据整理为“长表格”格式。`melt()` 和 `pivot()` 函数正是为此场景设计的。

Pandas 核心知识小结

本节要点

- DataFrame = 二维表格；Series = 一维序列
- **创建**：从 list、dict 或读取 CSV/Excel 文件
- **访问**：[]（按列名/条件）、iloc（按位置）、loc（按标签）
- **合并**：concat（直接按行列拼接）vs merge（SQL 风格的连接）
- **清洗**：drop_duplicates、dropna、fillna
- **变形**：melt（宽→长）、pivot（长→宽）

多维数组（NumPy）

二维表（Pandas）

Matplotlib 绘图库

SciPy 科学计算

Scikit-learn 机器学习

Scikit-image 图像处理

本章小结

Matplotlib 简介

Matplotlib 是 Python 最经典的二维科学绘图库，可绘制折线图、散点图、柱状图、热力图等。

核心对象层次

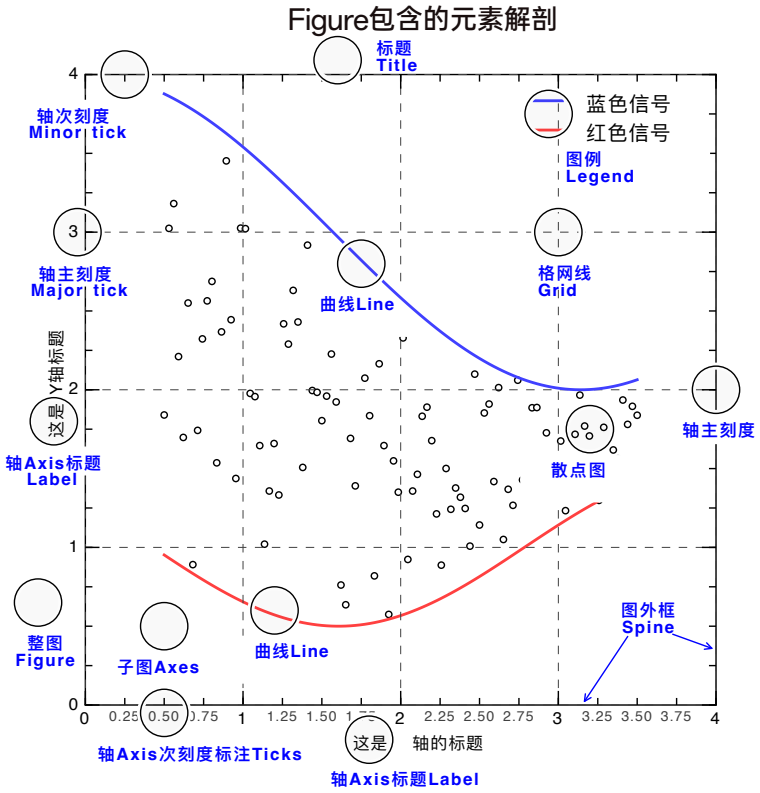
- `Figure`：整个图形画布
- `Axes`：子图（一个 Figure 可含多个）
- `Axis`：坐标轴（X 轴、Y 轴）
- `Title`：图标题

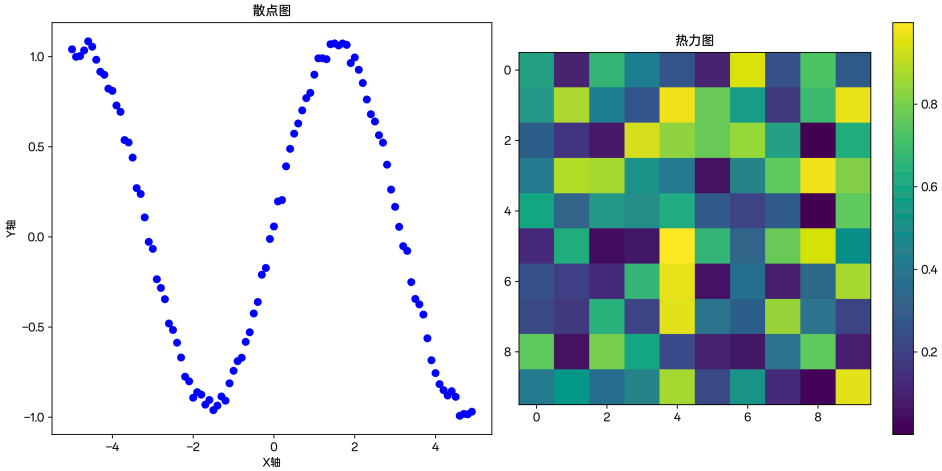
两种编程接口

- `pyplot` **接口**（state-based）：类似 MATLAB，简洁快捷，适合交互探索
- **面向对象接口**：更精细的控制，适合复杂绘图场景

```
conda install -c conda-forge matplotlib
```

Matplotlib 核心概念





一个 Figure 中包含的所有可视化元素

多个子图构成一个 Figure

pyplot 接口绘图

使用 pyplot 接口快速绘制折线图：

```python
import matplotlib.pyplot as plt
import numpy as np

x = np.arange(0, 2 * np.pi, 0.1)
y = np.sin(x)

plt.plot(x, y)                # 绘制折线图
plt.title('正弦函数')          # 设置图名
plt.xlabel('X 轴')            # X 轴标签
plt.ylabel('Y 轴')            # Y 轴标签
plt.grid(True)                # 显示网格
plt.show()                    # 展示图像
```

推荐风格：混合使用 pyplot 和面向对象接口——用 plt.subplot() 创建 Axes，再通过 ax 对象进行精细控制。

面向对象接口与多子图

```python
import matplotlib
import matplotlib.pyplot as plt
import numpy as np

matplotlib.rc("font", family='OPPO Sans')  # 设置中文字体

x = np.arange(-5, 5, 0.1)
y = np.sin(x) + np.random.rand(x.size) * 0.1
data = np.random.rand(10, 10)

fig, axs = plt.subplots(1, 2, figsize=(12, 6))

# 左图: 散点图
axs[0].scatter(x, y, c='blue', marker='o')
axs[0].set_title('散点图')
axs[0].set_xlabel('X轴')

# 右图: 热力图
cax = axs[1].imshow(data, cmap='viridis')
fig.colorbar(cax, ax=axs[1])
axs[1].set_title('热力图')

plt.tight_layout()
plt.savefig('subplots.pdf', dpi=300)
plt.show()
```

使用 Basemap 绘制地图

Basemap 是 Matplotlib 的地图绘制扩展，提供地图投影和基础地理数据：

```python
import matplotlib.pyplot as plt
from mpl_toolkits.basemap import Basemap

# 创建正射投影地图，resolution='i' 为中分辨率
m = Basemap(projection='ortho', lon_0=0, lat_0=0,
            resolution='i')

m.drawcoastlines()    # 绘制海岸线
m.drawcountries()     # 绘制国界线
m.drawmapboundary(fill_color='dodgerblue')  # 海洋填色
m.fillcontinents(color='yellowgreen',
                 lake_color='lightblue')    # 陆地填色

plt.title("人类生存的地球")
plt.show()
```

Basemap 支持多种地图投影坐标系，可在指定投影下绘制点、线、面等地理要素，适合空间数据可视化。

Matplotlib 核心知识小结

本节要点

- 核心对象层次：`Figure` → `Axes` → `Axis`、`Title`
- `pyplot` **接口**：快速绘图，适合交互探索
- **面向对象接口**：精细控制，适合复杂图形
- **推荐风格**：用 `plt.subplot()` 创建 `Axes` 对象，再调用 `ax.xxx()` 方法绘图
- 中文显示：需手动设置 `matplotlib.rc("font", family='...')`
- **Basemap**：地图绘制扩展，支持多种地图投影和基础地理数据叠加

多维数组（NumPy）

二维表（Pandas）

Matplotlib 绘图库

SciPy 科学计算

Scikit-learn 机器学习

Scikit-image 图像处理

本章小结

SciPy 简介

SciPy 是构建在 NumPy 之上的开源数学工具与算法库：

`scipy.linalg`

线性代数（矩阵分解、方程组求解）

`scipy.fft`

快速傅里叶变换

`scipy.optimize`

最优化与方程求根

`scipy.signal`

信号处理

`scipy.interpolate`

插值与样条平滑

`scipy.spatial`

空间数据结构与算法

`scipy.integrate`

积分与常微分方程求解

`scipy.stats`

统计分布与函数

SciPy：线性方程组求解

以二元一次方程组为例：

$$\begin{cases} 7x + 2y = 8 \\ 4x + 5y = 10 \end{cases}$$

```python
from scipy import linalg
import numpy as np

a = np.array([[7, 2], [4, 5]])  # 系数矩阵
b = np.array([8, 10])           # 常数项

# 方法一：直接使用 solve() 求解
res = linalg.solve(a, b)
print(res)  # [0.74074074 1.40740741]

# 方法二：通过矩阵求逆求解
res = np.mat(linalg.inv(a)) * np.mat(b).H
print(res)  # [[0.74074074] [1.40740741]]
```

SciPy：Delaunay 三角网剖分

TIN（不规则三角网） 是构建数字高程模型（DEM）的重要数据结构，常使用 Delaunay 三角剖分算法构建。

Delaunay 三角剖分的两个准则

- **空圆特性**：任意三角形的外接圆内，不包含其他点
- **最大化最小角**：所形成的三角形中最小内角最大（最接近规则三角形）

```python
import numpy as np
from scipy.spatial import Delaunay
import matplotlib.pyplot as plt

points = np.random.rand(20, 2)  # 随机生成 20 个二维点
tri = Delaunay(points)          # 构建 Delaunay 三角网

plt.triplot(points[:,0], points[:,1], tri.simplices)
plt.plot(points[:,0], points[:,1], 'o')
plt.show()
```

多维数组（NumPy）

二维表（Pandas）

Matplotlib 绘图库

SciPy 科学计算

Scikit-learn 机器学习

Scikit-image 图像处理

本章小结

Scikit-learn 简介

scikit-learn 是 Python 最流行的通用机器学习库，基于 NumPy、SciPy 构建。

分类与回归

线性回归、SVM、决策树、随机森林、神经网络等

聚类与降维

K-Means 聚类、PCA 主成分分析等

模型评估

交叉验证、精度评价指标（R^2、F1 等）

标准工作流程

数据收集 → 数据预处理 → 选择模型 → 模型训练 → 精度评价 → 模型预测

不同模型使用**统一的 API 接口**（`fit` / `predict` / `score`），可轻松切换模型进行对比。

Scikit-learn：线性回归实例

西北大学

```python
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import make_pipeline

# 准备训练数据（x 与 y 的关系，y 含噪声）
x = np.array([[1, 1], [1, 2], [2, 2], [2, 3]])
param = np.array([1, 2])
y = np.dot(x, param) + np.random.rand(4)

# 构建 Pipeline: 先标准化，再线性回归
model = make_pipeline(StandardScaler(), LinearRegression())
model.fit(x, y)                    # 训练

# 模型精度评估（R² 决定系数）
print(f'R² = {model.score(x, y):.2f}')  # 约 0.99

# 模型预测
ins = np.array([[3, 5]])
pred = model.predict(ins)
print(f'预测值: {pred}, 真实值: {np.dot(ins, param)}')
```

多维数组（NumPy）

二维表（Pandas）

Matplotlib 绘图库

SciPy 科学计算

Scikit-learn 机器学习

Scikit-image 图像处理

本章小结

Scikit-image 简介

Scikit-image 是基于 NumPy 和 SciPy 的开源数字图像处理库。

`skimage.io`

图像/视频的读取、保存和显示

`skimage.morphology`

形态学操作：膨胀、腐蚀等

`skimage.color`

图像颜色空间变换（RGB、灰度等）

`skimage.exposure`

亮度、对比度调整、直方图均衡

`skimage.filters`

图像滤波：增强、边缘检测、自动阈值

`skimage.feature`

图像特征检测与提取

`skimage.transform`

图像变换：旋转、拉伸、霍夫变换

`skimage.segmentation`

图像分割、等高线提取

Scikit-image：图像加载与边缘提取

西北大学

```python
import matplotlib.pyplot as plt
from skimage import io, feature
from skimage.color import rgb2gray

# 加载经典 Lenna 测试图像
image = io.imread('http://www.lenna.org/lena_std.tif')

# RGB 图像 → 灰度图像（ndarray 从 3D 变为 2D）
grayscale = rgb2gray(image)

# Canny 算子：提取图像边缘轮廓
profile = feature.canny(grayscale, sigma=1.0)

fig, ax = plt.subplots(1, 2, figsize=(12, 6))
ax[0].imshow(image)
ax[0].set_title('原始图像')
ax[0].axis('off')
ax[1].imshow(profile, plt.cm.grey)
ax[1].set_title('Canny 边缘提取')
ax[1].axis('off')
plt.tight_layout()
plt.show()
```

Scikit-image：等高线提取

西北大学

```python
import numpy as np
import matplotlib.pyplot as plt
from skimage import measure

# 用 ogrid 生成二维坐标，复数步长表示等分点数
x, y = np.ogrid[-np.pi:np.pi:100j, -np.pi:np.pi:100j]
r = np.sin(np.exp((np.sin(x)**3 + np.cos(y)**2)))

# 寻找灰度图像中的等高线（level 为灰度阈值）
contours = measure.find_contours(r, level=0.8)

fig, ax = plt.subplots()
ax.imshow(r, cmap=plt.cm.gray)  # 显示灰度底图

for contour in contours:
    ax.plot(contour[:, 1], contour[:, 0], linewidth=2)

ax.set_xticks([])
ax.set_yticks([])
plt.show()
```

`find_contours` 返回一个列表，每个元素是一条等高线的坐标序列 `(row, column)`。修改 `level` 参数可控制提取等高线的数量。

多维数组（NumPy）

二维表（Pandas）

Matplotlib 绘图库

SciPy 科学计算

Scikit-learn 机器学习

Scikit-image 图像处理

本章小结

科学计算生态全景

西北大学

核心基础库

- **NumPy**：多维数组 `ndarray`，科学计算基石
- **Pandas**：二维表格 `DataFrame`，数据处理利器
- **Matplotlib**：二维科学绘图库，可视化基础

高级扩展库

- **SciPy**：数学工具和算法（线性代数、插值、优化…）
- **Scikit-learn**：通用机器学习库
- **Scikit-image**：数字图像处理库

其他常用绘图库

- **Seaborn**：基于 Matplotlib，提供更丰富统计图形
- **Plotly**：动态交互式绘图，适合探索性分析

其他图像处理库

- **Pillow**：基础数字图像读取和处理
- **OpenCV**：复杂计算机视觉算法（C++ 绑定）

本章知识要点回顾

核心内容

- **NumPy**：`ndarray` 的创建、索引切片、合并分割、向量化编程、MaskedArray
- **Pandas**：`DataFrame` 读写、按行列名访问、concat/merge 合并、数据清洗
- **Matplotlib**：pyplot 接口 vs 面向对象接口、多子图、中文字体、Basemap 地图
- **SciPy**：线性方程组求解、Delaunay 三角剖分（TIN/DEM 相关）
- **Scikit-learn**：通用机器学习 Pipeline（标准化 → 建模 → 评估 → 预测）
- **Scikit-image**：图像加载、颜色空间转换、边缘检测、等高线提取

随堂练习

1. 设计一个函数，给定任意一张图像和任意一个卷积核，返回图像经过**卷积操作**后的结果。（提示：可使用 NumPy 向量化编程实现）
2. 读取 `laptop_price.csv` 文件，统计**每个品牌电脑的平均售价**，并使用 Matplotlib 绘制**柱状图**进行结果展示。
3. （思考题）说明 `np.stack()` 和 `np.concatenate()` 的核心区别，并各举一个在遥感数据处理中的应用场景。